

AIRA – Autonomous Manufacturing Intelligence Platform

Comprehensive Project Report

Project Overview

AIRA is a full-stack Autonomous Manufacturing Decision Platform that combines Next.js 15, FastAPI, LangGraph, Fireworks AI, AMD Developer Cloud, Digital Twin simulation, MCP enterprise integration, Event Streaming, Explainable AI and secure approval workflows. The platform continuously monitors enterprise systems, reasons over events, predicts outcomes, recommends actions and executes approved decisions.

Objectives

Build an enterprise-grade autonomous manufacturing platform with dashboard-first UX, AI Copilot, specialized AI agents, digital twin simulation, explainable AI, secure MCP integration, audit logging and real-time event processing.

Technology Stack

Frontend: Next.js 15, React, TypeScript, Tailwind CSS, Framer Motion, Recharts, Lucide React. Backend: FastAPI, Python, LangGraph, LangChain, Pydantic, WebSockets, SSE. AI: Fireworks AI Llama 3.3 70B. Compute: AMD Developer Cloud, ROCm, MI300X GPUs, vLLM. Data: TimescaleDB, pgvector, RAG. Integration: MCP Server, MCP Client, JWT, RBAC. Streaming: Kafka/Redpanda. Simulation: SimPy.

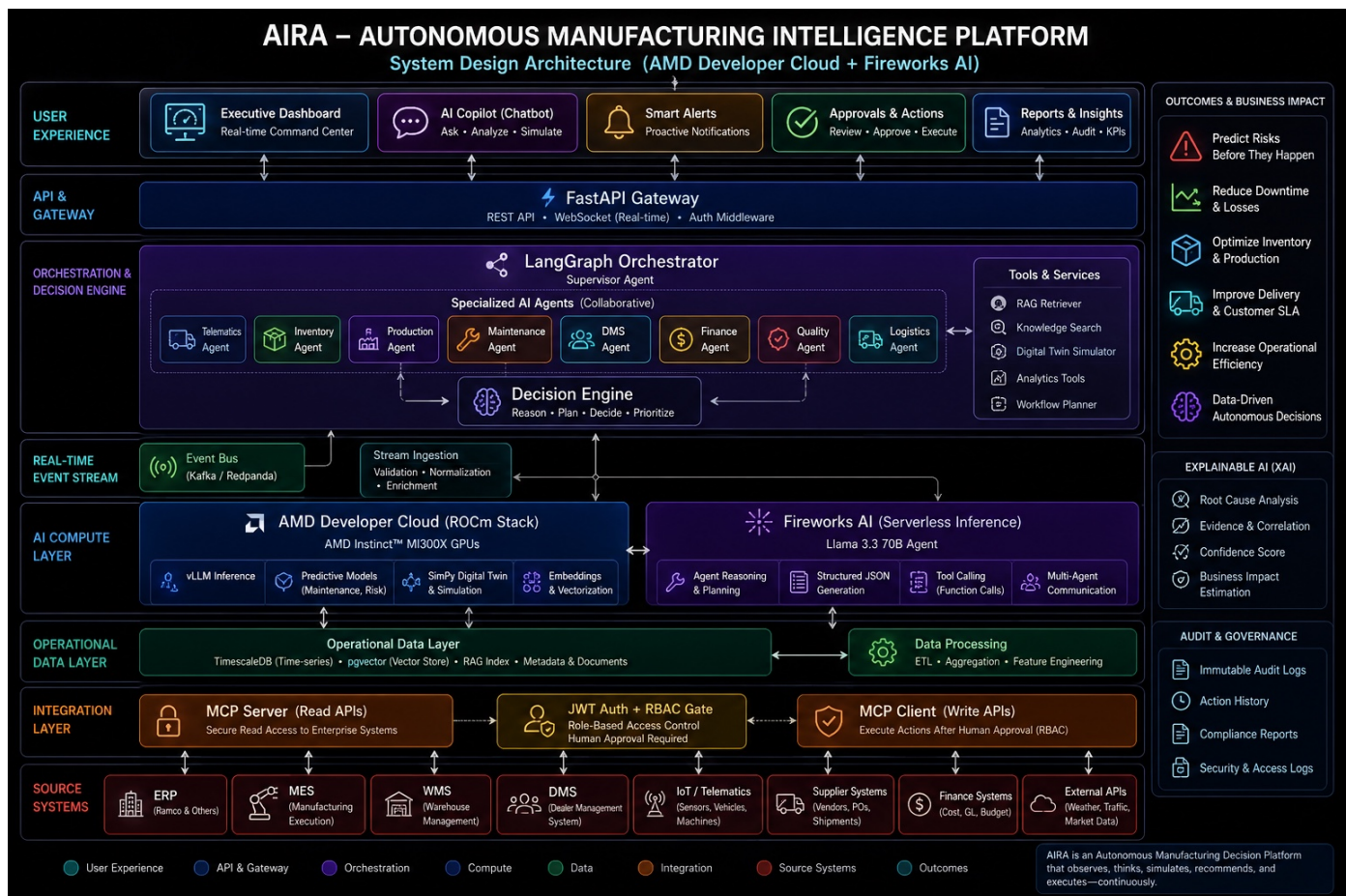
Implementation Details

The project was developed layer-by-layer. The frontend includes Executive Dashboard, AI Copilot, Smart Alerts, Approvals, Reports, animated Agent Canvas, Digital Twin panel, MCP Terminal, charts, glassmorphism UI, animations, icons, typewriter effects, API proxy routes, responsive layouts, hardware badges and WebSocket updates. The backend provides REST APIs, WebSockets, LangGraph orchestration, supervisor agent, eight specialized agents, decision engine, digital twin simulator, structured outputs, configuration management, audit logging and approval workflows.

Small Tools Used

Tailwind utilities, CSS variables, Google Fonts, SVG gauges, Lucide icons, Framer Motion, Recharts, Markdown rendering, environment variables, JSON schemas, TypeScript interfaces, mock datasets, Git, GitHub, npm, uvicorn, virtual environments, constants, configuration files and utility hooks were all used throughout the project.

System Design Architecture



Architecture Explanation

The architecture follows a layered approach. User Experience communicates with FastAPI Gateway. LangGraph Supervisor coordinates the Decision Engine and specialized agents (Telematics, Inventory, Production, Maintenance, DMS, Finance, Quality and Logistics). Event Bus continuously streams enterprise events. Fireworks AI performs reasoning, planning and tool calling while AMD Developer Cloud executes predictive models, vector embeddings and Digital Twin simulations. Operational Data Layer stores time-series, vector embeddings and metadata. MCP Server securely reads enterprise systems while MCP Client performs RBAC-controlled write operations after human approval. Business outcomes include predictive maintenance, reduced downtime, optimized inventory, explainable AI and governance.

Demo Workflow

Truck battery anomaly → Event Bus → LangGraph → Specialized Agents → Fireworks AI → AMD Digital Twin → Decision Engine → Recommendation → Human Approval → MCP Write → ERP/MES/DMS Updates → Audit Logs → Dashboard Health Improvement.

Note

This PDF is a concise report. A complete software design document with 80–120 pages covering architecture, APIs, workflows, sequence diagrams, component design, deployment, testing, and implementation details can also be produced.